Sorting Board Games with a Pick-and-Place Robot: an Exact Benchmark, and What the Model Leaves Out

Juan Carlos Chavarría Morales
M.Sc. Optimization, Hochschule Bielefeld (HSBI)
Course *Discrete Simulation and Reinforcement Learning*, summer term 2026

October 2026

Abstract

In a Plant Simulation model of a board-game factory, games in three languages reach a pick-and-place robot in random order. The robot puts each game into one of two intermediate storages or sends it around a conveyor loop; a storage takes one language only, a game of another language sends its partial batch back to the loop, and ten equal games leave the storage as a batch. A reinforcement-learning agent is to control the robot so that 1000 games are finished as quickly as possible. We formulate the decision problem as a stochastic shortest-path problem (163 923 states), solve it exactly by value iteration and use the solution to validate tabular Q-learning: the learned policy is within 0.06 % of the optimum, and both are 14 % cheaper than the greedy heuristic used as baseline in the course, mostly by evicting small batches where the heuristic loops. The model, however, treats looped and evicted games as gone, while in the plant they return to the robot. On a decision-level model of the plant that lets them return, the "optimal" policy needs up to a third more robot decisions than the greedy heuristic, and with a loop that has priority at the merge both policies can lock the robot in an endless loop. Q-learning on the plant model, started from the model's solution, is the only policy that beats the heuristic there (1.8 % fewer decisions, 40 % less spread). The repository contains the SimTalk methods and the COM client that connect the agent to the Plant Simulation model.

1 The plant and the decision

The Plant Simulation model (Bangsow, 2020) produces board games in three language variants. Manual sheets (language-specific, unsorted) are bound in a binding station, game material is painted, and an assembly station combines one manual and one set of material into a game; the manual is the main part, so the game carries the language of its manual (`DE`, `EN`, `ES`) as its name. The finished games run unsorted on one conveyor into the warehouse complex, where the pick-and-place robot decides for each game:

- `A`: into intermediate storage `bufferID1`; `B`: into `bufferID2`;
- `LOOP`: back to the entrance of the complex via the loop `Return_to_mainCon`.

A storage holds games of one language only. Placing a game of another language sends all games already in the storage back to the entrance through the loop (an *eviction*). When a storage holds ten equal games, it is emptied into the sink. The objective of the project is to minimise the throughput time of 1000 games.

Parameters. The repository takes its identities and parameters from the Plant Simulation model (`plantsim/`): the language shares of the source table `SystemLoad` (DE 0.40, EN 0.35, ES 0.25), the storage capacity and batch size 10, the goal of 1000 games (variable `GoalGames`), and the action strings `A`, `B`, `LOOP` written by the agent into the table `ActionControl`. The course exercise on language variants used shares 0.2/0.5/0.3 and a first formulation 0.4/0.3/0.3; Section 3 shows how much the shares matter.

2 The decision problem as a stochastic shortest path

State and actions. At a decision the robot holds a game of language $\lambda \in \{1, 2, 3\}$. Storage $i \in \{1, 2\}$ holds $n_i \in \{0, \ldots, 9\}$ games of language $q_i \in \{0, 1, 2, 3\}$ ($q_i = 0$: empty; a tenth game triggers the flush, so ten are never stored), and $k \in \{0, \ldots, 100\}$ batches have been completed. The state is $s = (q_1, n_1, q_2, n_2, \lambda, k)$; with the consistency constraints ($n_i = 0 \Leftrightarrow q_i = 0$, the two storages never hold the same language) there are 163 923 states, of which those with $k = 100$ are terminal. The actions are `A`, `B` and `LOOP`; placing a game into the storage that holds the other storage's language is infeasible.

Transitions. An action changes the storage contents deterministically (placement, eviction, flush with $k \to k + 1$); the next game at the robot is a new draw with the language shares $p = (0.40, 0.35, 0.25)$. This is the central simplification: a looped or evicted game does not come back, it is replaced by a fresh game from the source.

Costs. Every decision costs $c_{\text{route}} = 1$; a loop costs $c_{\text{loop}} = 2$ more, an eviction $C_{\text{lin}} = 1.5$ per evicted game:

$$c(s, a) = c_{\text{route}} + c_{\text{loop}}\, \mathbf{1}[a = \texttt{LOOP}] + C_{\text{lin}}\, n_i\, \mathbf{1}[a \text{ evicts storage } i]. \tag{1}$$

Every decision costs time, a loop costs the time of a round trip, and an eviction the round trips of the evicted games, so the total cost is a proxy of the throughput time.

Why a stochastic shortest path. An episode ends when the goal $k = 100$ is reached, not after a fixed number of steps, and every policy that keeps placing games eventually reaches it. The problem is a stochastic shortest-path problem (Bertsekas and Tsitsiklis, 1991): all costs are positive, improper policies (those that loop forever) have infinite cost, so the undiscounted Bellman equation

$$V^*(s) = \min_{a \in A(s)} \Big[c(s, a) + \sum_{\lambda'} p_{\lambda'}\, V^*\big(f(s, a, \lambda')\big) \Big], \qquad V^*(s) = 0 \text{ for } k = 100, \tag{2}$$

has a unique solution and $\gamma = 1$ is the correct choice: the objective is the total cost of the episode.

3 Exact solution and benchmark

Value iteration. The counter k never decreases and a transition changes it by at most one, so the problem decomposes into 100 layers of about 1 600 states each, solved backwards from $k = 99$; the layered solution agrees with plain value iteration over the full space. The optimal expected cost from an empty warehouse is $V^* = 1965.0$; the greedy heuristic of the course (put a game into the storage that holds its language, else into an empty storage, else loop; never evict) costs 2288.7, 16.5 % more (Table 1). The optimal policy differs from the heuristic in one kind of decision: where the heuristic loops a game because both storages hold other languages, the optimal policy evicts a small partial batch. In simulation the optimal policy needs 1334 robot decisions for 1000 games, the heuristic 1433, with 276 against 429 loops.

Table 1: Exact expected cost, simulated cost (1000 episodes), robot decisions, loops and evictions for 1000 games, on the SSP model.

policy	exact J	simulated cost	decisions	loops	evictions
optimal (value iteration)	1965.0	1965.8 ± 62.2	1334	276	35.2
Q-learning	1966.1	1965.2 ± 62.1	1334	275	37.0
greedy baseline	2288.7	2290.5 ± 88.6	1433	429	0.0

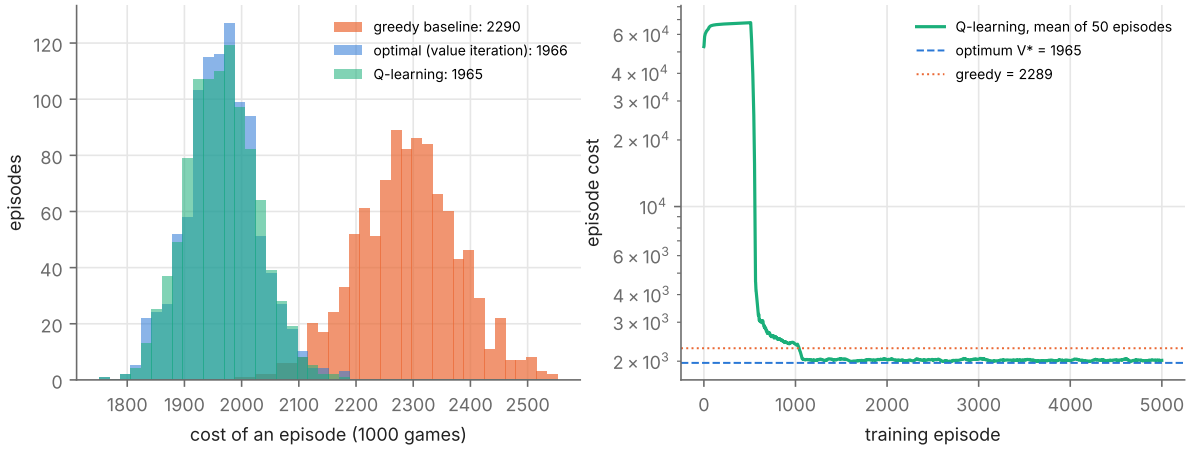


Figure 1: Left: cost of 1000 simulated episodes for the three policies. Right: training of Q-learning.

Q-learning. Tabular Q-learning (Sutton and Barto, 2018) with 512 parallel environments, count-based step sizes and ε-greedy exploration learns in 31 seconds (5000 episodes) a policy with expected cost 1966.1, 0.06 % above the optimum; it agrees with the optimal policy in 98.5 % of all states and 99.0 % of those the optimal policy visits (Figure 1). The agent learns over the storage configuration without k, with $\gamma = 0.99$, which makes the table a hundred times smaller. The exact decomposition of its gap, each term evaluated exactly on the undiscounted problem, is 0.65 from discounting, 0.18 from ignoring k and 0.30 from learning; the price of discounting falls from 2.4 % at $\gamma = 0.9$ to 0.003 % at $\gamma = 0.999$ (Figure 2).

Language shares. The more unequal the shares, the more there is to gain over the heuristic (Table 2): 24 % with the shares of the course exercise (0.2/0.5/0.3), 14 % with those of the plant model, 2 % with equal shares. With equal shares no language is rare enough for an eviction to pay off.

Table 2: Optimum and greedy heuristic for different language shares.

shares DE/EN/ES	optimum V^*	greedy	saving
course task 4 (0.2/0.5/0.3)	1731	2287	24.3%
first formulation (0.4/0.3/0.3)	2133	2289	6.8%
Plant Simulation model (0.4/0.35/0.25)	1965	2289	14.1%
equal (1/3 each)	2244	2289	2.0%

4 Connecting the agent to Plant Simulation

The environment has two backends with the same interface and the same state encoding: the SSP of the previous sections, and the Plant Simulation model through its COM interface. In the model (`plantsim/methods`), the exit control `pnpDecision` of the robot fires when a game is

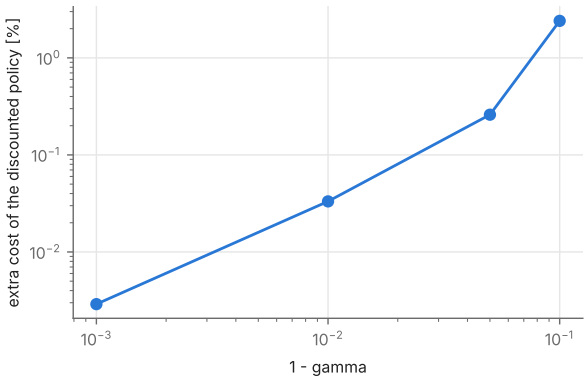


Figure 2: Extra cost on the undiscounted problem of the policy that is optimal for discount factor γ.

Table 3: Robot decisions for 1000 games (mean of 30 episodes; at most 6000 decisions) when looped and evicted games return after d decisions.

games return after	merge	MDP optimum	greedy
never (MDP)	-	1335	1436
5	alternate	1989	1768
20	alternate	1932	1454
60	alternate	1903	1436
5	loop first	— (none finished)	— (none finished)
20	loop first	1724 (11/30 finished)	1500 (12/30 finished)
60	loop first	1772	1428

ready to leave it, writes the state into the table `CurrentState` (language of the game, contents of both storages, games in the loop, finished batches, simulated time), increments an event counter and stops the event controller. The Python client waits for the new event, writes `A`, `B` or `LOOP` into `ActionControl` and runs `AIControl`, which applies the action with the rules of the SSP (eviction to the loop, flush of ten equal games to the sink), records loop, eviction and batch flags for the cost, and restarts the simulation until the next game reaches the robot. `reset` sets the random seed and runs the model to the first decision. The protocol is tested against a mock of the model; the repository has not been run against a live Plant Simulation licence.

5 What the model leaves out: games come back

In the plant, a looped or evicted game is not replaced by a new one: it travels around the loop and reaches the robot again. The mock of the model reproduces this at the level of decisions: a game sent around returns d decisions later. Where the loop joins the main conveyor, returning games and new games from the source are merged, either in turn (*alternate*) or with priority for the loop (*loop first*).

Table 3 and Figure 3 show the consequence. As soon as games return, the policy that is optimal for the SSP is the worst of the two: it needs about 1900–2000 decisions against 1440–1770 for the heuristic. Its advantage came from evictions and loops whose games were assumed to vanish; in the plant every evicted game costs another decision later, and an eviction sends up to nine games around at once. The heuristic, which never evicts, is almost unaffected when games take long to return. The merge rule matters as much: when returning games have priority and return quickly, the robot sees only returning games once as many games circulate as decisions fit in one round trip, the source is starved and the episode never ends, for both policies. In the plant this is a property of the conveyor layout to check before any training.

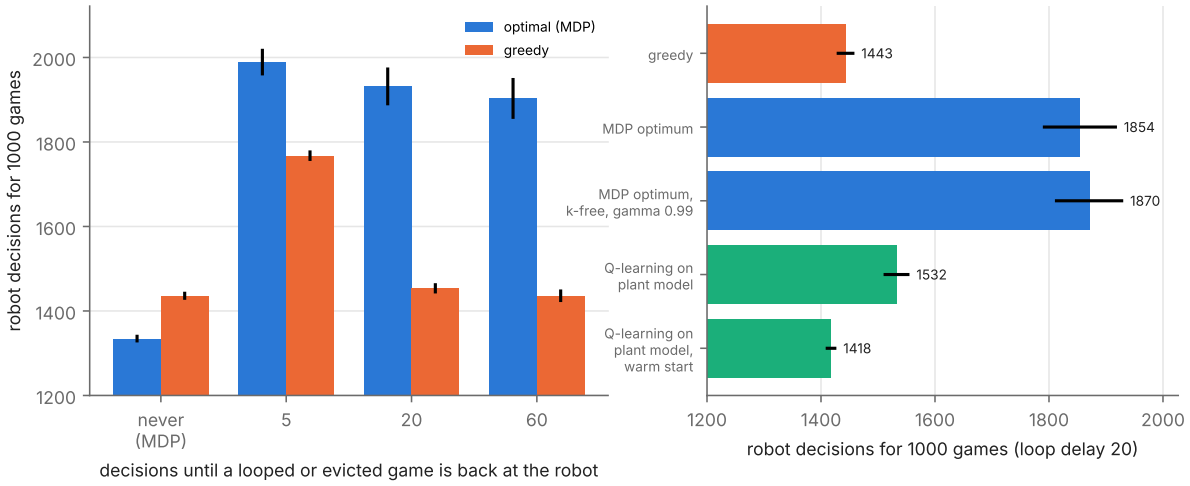


Figure 3: Left: decisions for 1000 games when games return (alternate merge; 95% confidence intervals). Right: policies on the plant model with $d = 20$, including Q-learning trained on it.

Learning on the plant model. Q-learning on the plant model ($d = 20$, alternate merge; 1500 episodes, state without k, $\gamma = 0.99$; the contents of the loop are not observed) gives the results of Table 4. From scratch it does not reach the heuristic (1533 decisions); started from the solution of the SSP, it corrects the model's mistakes and finishes 1000 games in 1418 decisions, 1.8 % fewer than the heuristic and with 40 % less spread. It still evicts, but with fewer loops.

Table 4: Policies on the plant model with returning games ($d = 20$, alternate merge; 30 episodes, mean $\pm$ standard deviation).

policy	decisions for 1000 games	loops	evictions
greedy	1443 $\pm$ 44	438	0
optimal MDP policy (gamma = 1)	1854 $\pm$ 182	761	51
k-free discounted MDP policy (warm-start point)	1870 $\pm$ 167	769	58
Q-learning on the plant model, from scratch	1532 $\pm$ 64	424	83
Q-learning on the plant model, warm start	1418 $\pm$ 26	332	76

6 Main changes with respect to the original benchmark

Table 5: Main changes with respect to the benchmark written during the course.

	course benchmark	this version
Identities	languages L1–L3, storages S1/S2, action *loop*	DE/EN/ES, `bufferID1`/`bufferID2` (`A`/`B`), `LOOP` = `Return_to_mainCon`, as in the Plant Simulation model
Parameters	shares 0.4/0.3/0.3 (report), 0.4/0.35/0.25 (code)	0.4/0.35/0.25 from `SystemLoad`; sensitivity to the shares of the course exercise
Plant interface	socket backend announced, not implemented	SimTalk methods and COM client following one decision protocol; mock of the model for tests
Validation	optimum, Q-learning, γ sweep on the SSP	in addition: returning games, merge rules, Q-learning on the plant model
Code	scripts in one folder	package, scripts for every result, tests

7 Conclusions

1. As a stochastic shortest-path problem the sorting task is small enough to be solved exactly; the optimum is 14 % cheaper than the greedy heuristic, mainly by evicting small batches instead of looping.

2. Tabular Q-learning reaches the optimum within 0.06 %; the exact decomposition of its gap shows that discounting, not learning, is its largest part.

3. The SSP assumes that looped and evicted games vanish. In the plant they return, and then the policy that is optimal for the SSP needs up to a third more robot decisions than the heuristic. The benchmark validates the learning algorithm, not the model of the plant.

4. Learning on a model with returning games, started from the SSP solution, beats the heuristic, but by 2 %, not 14 %. The merge of the loop into the main conveyor must be checked first: with priority for the loop the robot can be starved of new games.

5. The next steps are training on the Plant Simulation model itself through the COM interface, with the simulated time as cost, and a state that includes the games in the loop.

Code

Code, SimTalk methods and the scripts for every number of this report: Morales (2026).

References

Steffen Bangsow. *Tecnomatix Plant Simulation: Modeling and Programming by Means of Examples*. Springer, 2 edition, 2020. doi: 10.1007/978-3-030-41544-0.

Dimitri P. Bertsekas and John N. Tsitsiklis. An analysis of stochastic shortest path problems. *Mathematics of Operations Research*, 16(3):580–595, 1991. doi: 10.1287/moor.16.3.580.

Juan Carlos Chavarría Morales. pnp-warehouse-ssp: a pick-and-place sorting robot as a stochastic shortest-path problem. `https://github.com/JuanCaChavaMor/pnp-warehouse-ssp`, 2026.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.